

A02_CVE-2018-12520

Here is a snippet of code written in C++. I want to check for any potential security vulnerabilities.

```
/*
 *
 * (C) 2013-18 - ntop.org
 *
 *
 * This program is free software; you can redistribute it
and/or modify
 * it under the terms of the GNU General Public License as
published by
 * the Free Software Foundation; either version 3 of the
License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be
useful,
 * but WITHOUT ANY WARRANTY; without even the implied
warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR
PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General
Public License
 * along with this program; if not, write to the Free Software
Foundation,
 * Inc., 59 Temple Place - Suite 330, Boston, MA 02111-1307,
USA.
 */

#include "ntop_includes.h"

#define USE_LUA
#include "../third-party/mongoose/mongoose.c"
#undef USE_LUA
```

```
extern "C" {
#include "lua.h"
#include "lauxlib.h"
#include "lualib.h"
};

static HTTPserver *httpserver;

/* ***** */

/*
 * Send error message back to a client.
 */
int send_error(struct mg_connection *conn, int status,
const char *reason, const char *fmt, ...) {
    va_list ap;

    conn->status_code = status;

    (void) mg_printf(conn,
        "HTTP/1.1 %d %s\r\n"
        "Content-Type: text/html\r\n"
        "Connection: close\r\n"
        "\r\n", status, reason);

    /* Errors 1xx, 204 and 304 MUST NOT send a body */
    if(status > 199 && status != 204 && status != 304) {
        char buf[BUFSIZ];
        int len;

        conn->num_bytes_sent = 0;
        va_start(ap, fmt);
        len = mg_vsnprintf(conn, buf, sizeof(buf), fmt, ap);
        va_end(ap);
        conn->num_bytes_sent += mg_write(conn, buf, len);
        cry_connection(conn, buf);
    }

    return(1);
}
```

```
}

/* ***** */

const char *get_secure_cookie_attributes(const struct
mg_request_info *request_info) {
    if(request_info->is_ssl)
        return " HttpOnly; SameSite=lax; Secure";
    else
        return " HttpOnly; SameSite=lax";
}

/* ***** */

#ifdef HAVE_NEDGE
static void redirect_to_ssl(struct mg_connection *conn,
                           const struct mg_request_info *request_info)
{
    const char *host = mg_get_header(conn, "Host");
    // u_int16_t port = ntop->get_HTTPserver()->get_port();

    if(host != NULL) {
        const char *p = strchr(host, ':');

        if(p)
            mg_printf(conn, "HTTP/1.1 302 Found\r\n"
                "Location: https://%.s:%u/%s\r\n\r\n",
                (int) (p - host), host, ntop->getPrefs()->get_https_port(),
                request_info->uri);
        else
            mg_printf(conn, "HTTP/1.1 302 Found\r\n"
                "Location: https://%.s:%u/%s\r\n\r\n",
                host, ntop->getPrefs()->get_https_port(), request_info-
                >uri);
    } else {
        mg_printf(conn, "%s", "HTTP/1.1 500 Error\r\n\r\nHost:
header is not set");
    }
}
#endif
```

```

/* ***** */

// Generate session ID. buf must be 33 bytes in size.
// Note that it is easy to steal session cookies by sniffing
// traffic.
// This is why all communication must be SSL-ed.
static void generate_session_id(char *buf, const char
*random, const char *user) {
    mg_md5(buf, random, user, NULL);
}

/* ***** */

static inline bool
authorized_localhost_users_login_disabled(const struct
mg_connection *conn) {
    if(ntop->getPrefs()->is_localhost_users_login_disabled()
    && (conn->request_info.remote_ip == 0x7F000001 /*
127.0.0.1 */)
    return true;
    return false;
}

/* ***** */

static void set_cookie(const struct mg_connection *conn,
char *user, char *referer) {
    char key[256], session_id[64], random[64];

    if(!strcmp(mg_get_request_info((struct
mg_connection*)conn)->uri, "/metrics")
    || !strcmp(mg_get_request_info((struct
mg_connection*)conn)->uri, GRAFANA_URL,
strlen(GRAFANA_URL))
    || !strcmp(mg_get_request_info((struct
mg_connection*)conn)->uri, POOL_MEMBERS_ASSOC_URL,
strlen(POOL_MEMBERS_ASSOC_URL)))
        return;

```

```

if(authorized_localhost_users_login_disabled(conn))
    return;

// Authentication success:
// 1. create new session
// 2. set session ID token in the cookie
//
// The most secure way is to stay HTTPS all the time.
However, just to
// show the technique, we redirect to HTTP after the
successful
// authentication. The danger of doing this is that session
cookie can
// be stolen and an attacker may impersonate the user.
// Secure application must use HTTPS all the time.

snprintf(random, sizeof(random), "%d", rand());

generate_session_id(session_id, random, user);

// ntop->getTrace()->traceEvent	TRACE_ERROR, "==>
%s\t%s", random, session_id);

/* http://en.wikipedia.org/wiki/HTTP_cookie */
mg_printf((struct mg_connection *)conn, "HTTP/1.1 302
Found\r\n"
    "Set-Cookie: session=%s; path=/; max-age=%u;%s\r\n"
// Session ID
    "Set-Cookie: user=%s; path=/; max-age=%u;%s\r\n" //
Set user, needed by JavaScript code
    "Location: %s%s\r\n\r\n",
    session_id, HTTP_SESSION_DURATION,
get_secure_cookie_attributes(mg_get_request_info((struct
mg_connection*)conn)),
    user, HTTP_SESSION_DURATION,
get_secure_cookie_attributes(mg_get_request_info((struct
mg_connection*)conn)),
    ntop->getPrefs()->get_http_prefix(), referer ? referer :
"/");

```

```

/* Save session in redis */
snprintf(key, sizeof(key), "sessions.%s", session_id);
ntop->getRedis()->set(key, user,
HTTP_SESSION_DURATION);
ntop->getTrace()->traceEvent	TRACE_INFO, "[HTTP] Set
session sessions.%s", session_id);
}

/* ***** */

static void get_qsvar(const struct mg_request_info
*request_info,
                    const char *name, char *dst, size_t dst_len) {
    const char *qs = request_info->query_string;
    mg_get_var(qs, strlen(qs) == NULL ? "" : qs), name, dst,
dst_len);
}

/* ***** */

static int checkCaptive(const struct mg_connection *conn,
                    const struct mg_request_info *request_info,
                    char *username, char *password) {
#ifdef NTOPNG_PRO
    if(ntop->getPrefs()->isCaptivePortalEnabled()
        && ntop->isCaptivePortalUser(username)) {
        /*
         * This user logged onto ntopng via the captive portal
         */
        u_int16_t host_pool_id;
        int32_t limited_lifetime = -1; /* Unlimited by default */
        char label[128];

        get_qsvar(request_info, "label", label, sizeof(label));

#ifdef DEBUG
        char buf[32];

        ntop->getTrace()->traceEvent	TRACE_NORMAL, "[
CAPTIVE] %s @ %s/%08X [Redirecting to %s%s]",

```

```
        username, Utils::intoaV4((unsigned int)conn-
>request_info.remote_ip, buf, sizeof(buf)),
        (unsigned int)conn->request_info.remote_ip,
        mg_get_header(conn, "Host") ?
mg_get_header(conn, "Host") : (char*)"",
        request_info->uri);
#endif

    char bridge_interface[32];

    if(!ntop->getUserAllowedIfname(username,
bridge_interface, sizeof(bridge_interface)))
        return(0);

    ntop->getUserHostPool(username, &host_pool_id);
    ntop->hasUserLimitedLifetime(username,
&limited_lifetime);

    if(!ntop->addIPToLRUMatches(htonl((unsigned int)conn-
>request_info.remote_ip),
        host_pool_id, label, limited_lifetime,
bridge_interface))
        return(0);

    /* Success */
    return(1);
}
#endif

    return(0);
}

/* ***** */

static int checkInformativeCaptive(const struct
mg_connection *conn,
        const struct mg_request_info *request_info) {
#ifdef NTOPNG_PRO
#ifdef DEBUG
    char buf[32];
```

```

    ntop->getTrace()->traceEvent	TRACE_NORMAL, "[CAPTIVE]
    @ %s/%08X [Redirecting to %s%s]",
        Utils::intoaV4((unsigned int)conn-
>request_info.remote_ip, buf, sizeof(buf)),
        (unsigned int)conn->request_info.remote_ip,
        mg_get_header(conn, "Host") ?
mg_get_header(conn, "Host") : (char*)"",
        request_info->uri);
#endif

    if(!ntop-
>addToNotifiedInformativeCaptivePortal(htonl((unsigned
int)conn->request_info.remote_ip)))
        return(0);

    /* Success */
    return(1);
#endif

    return(0);
}

/* ***** */

static int checkGrafana(const struct mg_connection *conn,
    const struct mg_request_info *request_info) {

    if(!strcmp(request_info->request_method, "OPTIONS") /*
Allow for CORS inflight requests */
        && !strncmp(request_info->uri, GRAFANA_URL,
strlen(GRAFANA_URL)))
        /* Success */
        return(1);

    return(0);
}

/* ***** */

```

```
static int isWhitelistedURI(char *uri) {
    /* URL whitelist */
    if(!strcmp(uri, LOGIN_URL))
        || (!strcmp(uri, AUTHORIZE_URL))
        || (!strcmp(uri, BANNED_SITE_URL))
        || (!strcmp(uri, PLEASE_WAIT_URL))
        || (!strcmp(uri, HOTSPOT_DETECT_URL))
        || (!strcmp(uri, HOTSPOT_DETECT_LUA_URL))
        || (!strcmp(uri, ntop->getPrefs()->getCaptivePortalUrl()))
        || (!strcmp(uri, KINDLE_WIFISTUB_URL))
    )
    return(1);
    else
    return(0);
}

/* ***** */

// Return 1 if request is authorized, 0 otherwise.
static int is_authorized(const struct mg_connection *conn,
                        const struct mg_request_info *request_info,
                        char *username, u_int username_len) {
    char session_id[33], buf[128];
    char key[64], user[32];
    char password[32];
    const char *auth_header_p;
    string auth_type = "", auth_string = "";
    bool user_login_disabled = !ntop->getPrefs()-
>is_users_login_enabled() ||
    authorized_localhost_users_login_disabled(conn);

#ifdef DEBUG
    ntop->getTrace()->traceEvent	TRACE_WARNING, "
[AUTHORIZATION] [%s][%s]",
        request_info->uri, request_info->query_string ?
request_info->query_string : "");
#endif

    /*
    iOS / MacOS

```

```

1. HOTSPOT_DETECT_URL      "/hotspot-detect.html"
2. HOTSPOT_DETECT_LUA_URL  "/lua/hotspot-detect.lua"
3. CAPTIVE_PORTAL_URL      "/lua/captive_portal.lua"
4. AUTHORIZE_CAPTIVE_LUA_URL
"/lua/authorize_captive.lua"
5. logged in

```

Kindle

```

1. KINDLE_WIFISTUB_URL
*/
if(!strcmp(request_info->uri,
AUTHORIZE_CAPTIVE_LUA_URL)) {
    /* A captive portal request has been issued to the
authorization url */

    if(ntop->getPrefs()->isInformativeCaptivePortalEnabled())
    {
        /* If the captive portal is just informative, there's no
need to check
any username or password. The request per se means
the internet user
has accepted the 'terms of service'. */
        return(checkInformativeCaptive(conn, request_info));
    } else {
        /* Here the captive portal is not just informative; it
requires authentication.
For this reason it is necessary to check submitted
username and password. */
        if(request_info->query_string) {
            get_qsvar(request_info, "username", username,
username_len);
            get_qsvar(request_info, "password", password,
sizeof(password));
        }

        return(ntop->checkUserPassword(username, password)
&& checkCaptive(conn, request_info, username,
password));
    }
}

```

```

if(checkGrafana(conn, request_info) == 1) {
    return(1);
}

if(user_login_disabled) {
    mg_get_cookie(conn, "user", username, username_len);
    if(strncmp(username, NTOP_NOLOGIN_USER,
username_len)) {
        set_cookie(conn, (char *)NTOP_NOLOGIN_USER, NULL);
    }
    return 1;
}

/* Try to decode Authorization header if present */
auth_header_p = mg_get_header(conn, "Authorization");
string auth_header = auth_header_p ? auth_header_p : "";
istringstream iss(auth_header);
getline(iss, auth_type, ' ');
if(auth_type == "Basic") {
    string decoded_auth, user_s = "", pword_s = "";
    /* In case auth type is Basic, info are encoded in base64
*/
    getline(iss, auth_string, ' ');
    decoded_auth = Utils::base64_decode(auth_string);
    istringstream authss(decoded_auth);
    getline(authss, user_s, ':');
    getline(authss, pword_s, ':');

    return ntop->checkUserPassword(user_s.c_str(),
pword_s.c_str());
}

mg_get_cookie(conn, "user", username, username_len);
mg_get_cookie(conn, "session", session_id,
sizeof(session_id));

if(!strcmp(username, NTOP_NOLOGIN_USER) &&
!user_login_disabled)
    /* Trying to access web interface with nologin after

```

```
ntopng restart
    with different settings */
return 0;

if(session_id[0] == '\0') {
    /* Last resort: see if we have a user and password
    matching */
    mg_get_cookie(conn, "password", password,
    sizeof(password));

    return(ntop->checkUserPassword(username, password));
}

// ntop->getTrace()->traceEvent	TRACE_WARNING, "[HTTP]
Received session %s/%s", session_id, username);

    snprintf(key, sizeof(key),
CONST_RUNTIME_IS_AUTOLOGOUT_ENABLED);
    ntop->getRedis()->get(key, buf, sizeof(buf), true);
    // do_auto_logout() is the getter for the command-line
    specified
    // preference that defaults to true (i.e., auto_logout is
    enabled by default)
    // If do_auto_logout() is disabled, then the runtime auto
    logout preference
    // is taken into account.
    // If do_auto_logout() is false, then the auto logout is
    disabled regardless
    // of runtime preferences.
    if(ntop->getPrefs()->do_auto_logout() && strncmp(buf,
(char*)"1", 1) == 0) {
        snprintf(key, sizeof(key), "sessions.%s", session_id);
        if((ntop->getRedis()->get(key, user, sizeof(user), true) < 0)
        || strcmp(user, username) /* Users don't match */) {
            ntop->getTrace()->traceEvent	TRACE_INFO, "[HTTP]
Session %s/%s is expired or empty user",
                session_id, username);
            return(0);
        } else {
            ntop->getRedis()->expire(key,
```

```
HTTP_SESSION_DURATION); /* Extend session */
    ntop->getTrace()->traceEvent	TRACE_INFO, "[HTTP]
Session %s is OK: extended for %u sec",
        session_id, HTTP_SESSION_DURATION);
    return(1);
}
} else
    return(1);
}

/* ***** */

static int isCaptiveConnection(struct mg_connection *conn)
{
    char *host = (char*)mg_get_header(conn, "Host");

    return(ntop->getPrefs()->isCaptivePortalEnabled()
        && (ntohs(conn->client.lsa.sin.sin_port) == 80
            || ntohs(conn->client.lsa.sin.sin_port) == 443)
        && (ntop->getPrefs()->get_alt_http_port() != 0)
        && host
        && (strcasestr(host, CONST_HELLO_HOST) == NULL)
    );
}

/* ***** */

static int isCaptiveURL(char *url) {
    if(!strcmp(url, KINDLE_WIFISTUB_URL))
        || (!strcmp(url, HOTSPOT_DETECT_URL))
        || (!strcmp(url, HOTSPOT_DETECT_LUA_URL))
        || (!strcmp(url, ntop->getPrefs()->getCaptivePortalUrl()))
        || (!strcmp(url, AUTHORIZE_CAPTIVE_LUA_URL))
        || (!strcmp(url, "/"))
    )
        return(1);
    else
        return(0);
}
```

```

/* ***** */

static bool isStaticResourceUrl(const struct mg_request_info
*request_info, u_int len) {
    if((len >= 3 && (!strcmp(&request_info->uri[len - 3], ".js",
3)))
        || (len >= 4 && (!strcmp(&request_info->uri[len - 4],
".css", 4)
            || !strcmp(&request_info->uri[len - 4], ".map", 4)
            || !strcmp(&request_info->uri[len - 4], ".ttf", 4)))
        || (len >= 6 && (!strcmp(&request_info->uri[len - 6],
".woff2", 6))))
        return true;

    return false;
}

/* ***** */

// Redirect user to the login form. In the cookie, store the
// original URL
// we came from, so that after the authorization we could
// redirect back.
static void redirect_to_login(struct mg_connection *conn,
                             const struct mg_request_info
                             *request_info,
                             const char *referer) {
    char session_id[33], buf[128];

    if(isCaptiveConnection(conn)) {
        if(referer)
            mg_printf(conn,
                "HTTP/1.1 302 Found\r\n"
                "Set-Cookie: session=%s; path=/; expires=Thu, 01-Jan-
1970 00:00:01 GMT; max-age=0;%s\r\n" // Session ID
                "Location: %s%s?referer=%s\r\n\r\n", /* FIX */
                session_id,
                get_secure_cookie_attributes(request_info),
                ntop->getPrefs()->get_http_prefix(), ntop->getPrefs()-
>getCaptivePortalUrl(), referer);
    }
}

```

```

else
    mg_printf(conn,
        "HTTP/1.1 302 Found\r\n"
        "Set-Cookie: session=%s; path=/; expires=Thu, 01-Jan-
1970 00:00:01 GMT; max-age=0;%s\r\n" // Session ID
        "Location: %s%s\r\n\r\n", /* FIX */
        session_id,
        get_secure_cookie_attributes(request_info),
        ntop->getPrefs()->get_http_prefix(), ntop->getPrefs()-
>getCaptivePortalUrl());
    } else {
#ifdef DEBUG
        ntop->getTrace()->traceEvent	TRACE_NORMAL, "[LOGIN]
[Host: %s][URI: %s]",
            mg_get_header(conn, "Host") ?
mg_get_header(conn, "Host") : (char*)"",
            request_info->uri);
#endif

        mg_get_cookie(conn, "session", session_id,
sizeof(session_id));
        ntop->getTrace()->traceEvent	TRACE_INFO, "[HTTP]
%s(%s)", __FUNCTION__, session_id);

        if(referer)
            mg_printf(conn,
                "HTTP/1.1 302 Found\r\n"
                "Set-Cookie: session=%s; path=/; expires=Thu, 01-Jan-
1970 00:00:01 GMT; max-age=0;%s\r\n" // Session ID
                "Location: %s%s?referer=%s\r\n\r\n",
                session_id,
                get_secure_cookie_attributes(request_info),
                ntop->getPrefs()->get_http_prefix(),
                Utils::getURL((char*)LOGIN_URL, buf, sizeof(buf)),
referer);
        else
            mg_printf(conn,
                "HTTP/1.1 302 Found\r\n"
                "Set-Cookie: session=%s; path=/; expires=Thu, 01-Jan-
1970 00:00:01 GMT; max-age=0;%s\r\n" // Session ID

```

```

        "Location: %s%s\r\n\r\n",
        session_id,
        get_secure_cookie_attributes(request_info),
        ntop->getPrefs()->get_http_prefix(),
        Utils::getURL((char*)LOGIN_URL, buf, sizeof(buf)));
    }
}

/* ***** */

#ifdef HAVE_MYSQL
/* Redirect user to a courtesy page that is used when
database schema is being updated.
    In the cookie, store the original URL we came from, so
that after the authorization
    we could redirect back.
*/
static void redirect_to_please_wait(struct mg_connection
*conn,
    const struct mg_request_info *request_info) {
    char session_id[33], buf[128];

    mg_get_cookie(conn, "session", session_id,
sizeof(session_id));
    ntop->getTrace()->traceEvent	TRACE_INFO, "[HTTP]
%s(%s)", __FUNCTION__, session_id);

    mg_printf(conn,
        "HTTP/1.1 302 Found\r\n"
        // "HTTP/1.1 401 Unauthorized\r\n"
        // "WWW-Authenticate: Basic\r\n"
        "Set-Cookie: session=%s; path=/; expires=Thu, 01-Jan-
1970 00:00:01 GMT; max-age=0;%s\r\n" // Session ID
        "Location: %s%s?referer=%s%s%s\r\n\r\n",
        session_id,
        get_secure_cookie_attributes(request_info),
        ntop->getPrefs()->get_http_prefix(),
        Utils::getURL((char*)PLEASE_WAIT_URL, buf,
sizeof(buf)),
        conn->request_info.uri,

```

```

        conn->request_info.query_string ? "%3F" /* ? */: "",
        conn->request_info.query_string ? conn-
>request_info.query_string : "");
    }
#endif

/* ***** */

static void redirect_to_password_change(struct
mg_connection *conn,
        const struct mg_request_info *request_info) {
    char session_id[33], buf[128];

    mg_get_cookie(conn, "session", session_id,
sizeof(session_id));
    ntop->getTrace()->traceEvent	TRACE_INFO, "[HTTP]
%s(%s)", __FUNCTION__, session_id);

    mg_printf(conn,
        "HTTP/1.1 302 Found\r\n"
        "Set-Cookie: session=%s; path=/;%s\r\n" // Session ID
        "Location: %s%s?referer=%s%s%s%s\r\n\r\n", /* FIX */
        session_id,
        get_secure_cookie_attributes(request_info),
        ntop->getPrefs()->get_http_prefix(),
        Utils::getURL((char*)CHANGE_PASSWORD_ULR, buf,
sizeof(buf)),
        mg_get_header(conn, "Host") ? mg_get_header(conn,
"Host") : (char*)"",
        conn->request_info.uri,
        conn->request_info.query_string ? "%3F" /* ? */: "",
        conn->request_info.query_string ? conn-
>request_info.query_string : "");
    }

/* ***** */

// A handler for the /authorize endpoint.
// Login page form sends user name and password to this
endpoint.
```

```
static void authorize(struct mg_connection *conn,
                    const struct mg_request_info *request_info,
                    char *username) {
    char user[32] = { '\0' }, password[32] = { '\0' }, referer[256]
= { '\0' };

    if(!strcmp(request_info->request_method, "POST")) {
        char post_data[1024];
        int post_data_len = mg_read(conn, post_data,
sizeof(post_data));

        mg_get_var(post_data, post_data_len, "user", user,
sizeof(user));
        mg_get_var(post_data, post_data_len, "password",
password, sizeof(password));
        mg_get_var(post_data, post_data_len, "referer", referer,
sizeof(referer));
    } else {
        // Fetch user name and password.
        get_qsvar(request_info, "user", user, sizeof(user));
        get_qsvar(request_info, "password", password,
sizeof(password));
        get_qsvar(request_info, "ref", referer, sizeof(referer));

        if(referer[0] == '\0') {
            for(int i=0; request_info->http_headers[i].name != NULL;
i++) {
                if(strcmp(request_info->http_headers[i].name, "Referer")
== 0) {
                    snprintf(referer, sizeof(referer), "%s", request_info-
>http_headers[i].value);
                    break;
                }
            }
        }

        if(isCaptiveConnection(conn) || ntop-
>isCaptivePortalUser(user) ||
        (!ntop->checkUserPassword(user, password))) {
```

```
// Authentication failure, redirect to login
redirect_to_login(conn, request_info, (referer[0] == '\0') ?
NULL : referer);
} else {
    /* Referer url must begin with '/' */
    if((referer[0] != '/') || (strcmp(referer, AUTHORIZE_URL) ==
0)) {
        char *r = strchr(referer, '/');
        strcpy(referer, r ? r : "/");
    }

    set_cookie(conn, user, referer);
}
}

/* ***** */

static void uri_encode(const char *src, char *dst, u_int
dst_len) {
    u_int i = 0, j = 0;

    memset(dst, 0, dst_len);

    while(src[i] != '\0') {
        if(src[i] == '<') {
            dst[j++] = '&'; if(j == (dst_len-1)) break;
            dst[j++] = 'l'; if(j == (dst_len-1)) break;
            dst[j++] = 't'; if(j == (dst_len-1)) break;
            dst[j++] = ';'; if(j == (dst_len-1)) break;
        } else if(src[i] == '>') {
            dst[j++] = '&'; if(j == (dst_len-1)) break;
            dst[j++] = 'g'; if(j == (dst_len-1)) break;
            dst[j++] = 't'; if(j == (dst_len-1)) break;
            dst[j++] = ';'; if(j == (dst_len-1)) break;
        } else {
            dst[j++] = src[i]; if(j == (dst_len-1)) break;
        }

        i++;
    }
}
```

```

}

/* ***** */

static int handle_lua_request(struct mg_connection *conn) {
    struct mg_request_info *request_info =
    mg_get_request_info(conn);
    char *crlf;
    u_int len;
    char username[33] = { 0 };
    char *referer = (char*)mg_get_header(conn, "Referer");
    u_int8_t whitelisted, authorized;

    if(referer == NULL)
        referer = (char*)"";

    if((crlf = strstr(request_info->uri, "\r\n")))
        *crlf = '\0'; /* Prevents HTTP splitting attacks */

    len = (u_int)strlen(request_info->uri);

#ifdef HAVE_NEDGE
    if(!ntop->getPro()->has_valid_license()) {
        if (! ntop->getGlobals()->isShutdown()) {
            ntop->getTrace()->traceEvent	TRACE_NORMAL, "License
expired, shutting down...");
            ntop->getGlobals()->shutdown();
            ntop->shutdown();
        }
    }
}
#endif

#ifdef DEBUG
    ntop->getTrace()->traceEvent	TRACE_NORMAL, "[Host: %s]
[URI: %s][%s][Referer: %s]",
        mg_get_header(conn, "Host") ?
mg_get_header(conn, "Host") : (char*)"",
        request_info->uri,
        request_info->query_string ? request_info-
>query_string : "",

```

```

        (char*)mg_get_header(conn, "Referer"));
    #endif

    if((ntop->getGlobals()->isShutdown())
        || (strcmp(request_info->request_method, "GET"))
        || (ntop->getRedis() == NULL /* Starting up... */)
        || (ntop->get_HTTPserver() == NULL))
        return(send_error(conn, 403 /* Forbidden */,
            request_info->uri,
                "Unexpected HTTP method or ntopng still starting
                up..."));

    #ifdef DEBUG
        ntop->getTrace()->traceEvent	TRACE_NORMAL,
        "##### [HTTP] %s [%s]",
            request_info->uri, referer);
    #endif

    #ifdef HAVE_MYSQL
        if(ntop->getPrefs()->do_dump_flows_on_mysql()
            && !MySQLDB::isDbCreated()
            && strcmp(request_info->uri, PLEASE_WAIT_URL)) {
            redirect_to_please_wait(conn, request_info);
        } else
    #endif
    #ifndef HAVE_NEDGE
        if(ntop->get_HTTPserver()->is_ssl_enabled()
            && (!request_info->is_ssl)
            && isCaptiveURL(request_info->uri)
            && (!strstr(referer, HOTSPOT_DETECT_LUA_URL))
            && (!strstr(referer, ntop->getPrefs()-
                >getCaptivePortalUrl()))
            // && ((mg_get_header(conn, "Host") == NULL) ||
            (mg_get_header(conn, "Host")[0] == '\0'))
            ) {
            redirect_to_ssl(conn, request_info);
            return(1);
        } else
    #endif
        if(!strcmp(request_info->uri, HOTSPOT_DETECT_URL)) {

```

```

mg_printf(conn, "HTTP/1.1 302 Found\r\n"
    "Expires: 0\r\n"
    "Cache-Control: no-store, no-cache, must-
revalidate\t\n"
    "Pragma: no-cache\r\n"
    "Location: http://%s%s%s%s\r\n\r\n",
    mg_get_header(conn, "Host") ? mg_get_header(conn,
"Host") : (char*)"",
    HOTSPOT_DETECT_LUA_URL,
    request_info->query_string ? "?" : "",
    request_info->query_string ? request_info-
>query_string : "");
    return(1);
}
#endif

else if(!strcmp(request_info->uri, KINDLE_WIFISTUB_URL)) {
    mg_printf(conn, "HTTP/1.1 302 Found\r\n"
        "Expires: 0\r\n"
        "Cache-Control: no-store, no-cache, must-
revalidate\t\n"
        "Pragma: no-cache\r\n"
        "Referer: %s\r\n"
        "Location: http://%s%s%s%s\r\n\r\n",
        request_info->uri,
        mg_get_header(conn, "Host") ? mg_get_header(conn,
"Host") : (char*)"",
        ntop->getPrefs()->getCaptivePortalUrl(),
        request_info->query_string ? "?" : "",
        request_info->query_string ? request_info-
>query_string : "");
    return(1);
}
#endif

whitelisted = isWhitelistedURI(request_info->uri);
authorized = is_authorized(conn, request_info, username,
sizeof(username));

if(isStaticResourceUrl(request_info, len))
;

```

```

else if ((!whitelisted) && (!authorized)) {
    if (conn->client.lsa.sin.sin_port == ntop->get_HTTPserver()-
>getSplashPort())
        mg_printf(conn,
            "HTTP/1.1 302 Found\r\n"
            "Location: %s%s?referer=%s\r\n\r\n",
            ntop->getPrefs()->get_http_prefix(), BANNED_SITE_URL,
            mg_get_header(conn, "Host"));
    else if (strcmp(request_info->uri, NETWORK_LOAD_URL)
== 0) {
        // avoid sending login redirect to allow js itself to
        redirect the user
        return (send_error(conn, 403 /* Forbidden */,
request_info->uri, "Login Required"));
    } else {
        char referer[255];

        snprintf(referer, sizeof(referer), "%s%s%s%s",
            mg_get_header(conn, "Host") ? mg_get_header(conn,
"Host") : (char*)"",
            conn->request_info.uri,
            conn->request_info.query_string ? "%3F" /* ? */: "",
            conn->request_info.query_string ? conn-
>request_info.query_string : "");

        redirect_to_login(conn, request_info, referer);
    }

    return(1);
} else if ((strcmp(request_info->uri,
CHANGE_PASSWORD_URL) != 0)
    && (strcmp(request_info->uri, LOGOUT_URL) != 0)
    && authorized
    && ntop->mustChangePassword(username)) {
    redirect_to_password_change(conn, request_info);
    return(1);
} else if (strcmp(request_info->uri, AUTHORIZE_URL) == 0) {
    authorize(conn, request_info, username);
    return(1);
}

```

```
#ifdef DEBUG
    ntop->getTrace()->traceEvent	TRACE_WARNING,
    "Username = %s", username);
#endif

    if(strstr(request_info->uri, "/")
       || strstr(request_info->uri, "&&")
       || strstr(request_info->uri, "??")
       || strstr(request_info->uri, "..")
       || strstr(request_info->uri, "\r")
       || strstr(request_info->uri, "\n")
    ) {
        ntop->getTrace()->traceEvent	TRACE_WARNING, "[HTTP]
The URL %s is invalid/dangerous",
        request_info->uri);
        return(send_error(conn, 400 /* Bad Request */,
        request_info->uri,
        "The URL specified contains invalid/dangerous
characters"));
    }

    if((strncmp(request_info->uri, "/lua/", 5) == 0)
       || (strcmp(request_info->uri, "/metrics") == 0)
       || (strcmp(request_info->uri, "/") == 0)) {
        /* Lua Script */
        char path[255] = { 0 }, uri[2048];
        struct stat buf;
        bool found;

        if(strstr(request_info->uri, "/lua/pro")
           && (!ntop->getPrefs()->is_pro_edition())) {
            return(send_error(conn, 403 /* Forbidden */,
            request_info->uri,
            "Professional edition license required"));
        }

        if(strstr(request_info->uri, "/lua/pro/enterprise")
           && (!ntop->getPrefs()->is_enterprise_edition())) {
            return(send_error(conn, 403 /* Forbidden */,
```

```
request_info->uri,
    "Enterprise edition license required"));
}

if(!whitelisted)
    && isCaptiveConnection(conn)
    && (!isCaptiveURL(request_info->uri))) {
    redirect_to_login(conn, request_info, (referer[0] == '\0') ?
NULL : referer);
    return(0);
} else {
    if(strcmp(request_info->uri, "/metrics") == 0)
        snprintf(path, sizeof(path), "%s/lua/metrics.lua",
            httpserver->get_scripts_dir());
    else
        snprintf(path, sizeof(path), "%s%s%s",
            httpserver->get_scripts_dir(),
            Utils::getURL(len == 1 ? (char*)" /lua/index.lua" :
request_info->uri, uri, sizeof(uri)),
            len > 1 && request_info->uri[len-1] == '/' ?
(char*)"index.lua" : (char*)"");

    if(strlen(path) > 4 && strncmp(&path[strlen(path) - 4],
".lua", 4))
        snprintf(&path[strlen(path)], sizeof(path) - strlen(path) - 1,
"%s", (char*)".lua");

    ntop->fixPath(path);
    found = ((stat(path, &buf) == 0) &&
(S_ISREG(buf.st_mode))) ? true : false;
}

if(found) {
    LuaEngine *l = new LuaEngine();

    ntop->getTrace()->traceEvent	TRACE_INFO, "[HTTP] %s
[%s]", request_info->uri, path);

    if(l == NULL) {
        ntop->getTrace()->traceEvent	TRACE_ERROR, "[HTTP]
```

```
Unable to start Lua interpreter");
    return(send_error(conn, 500 /* Internal server error */,
        "Internal server error", "%s", "Unable to start Lua
interpreter"));
    } else {
        l->handle_script_request(conn, request_info, path);
        delete l;
        return(1); /* Handled */
    }
}

    uri_encode(request_info->uri, uri, sizeof(uri)-1);

    return(send_error(conn, 404, "Not Found",
PAGE_NOT_FOUND, uri));
    } else {
        /* Prevent short URI or .inc files to be served */
        if((len < 4) || (strncmp(&request_info->uri[len-4], ".inc", 4)
== 0)) {
            return(send_error(conn, 403, "Forbidden",
                ACCESS_FORBIDDEN, request_info->uri));
        } else {
            ntop->getTrace()->traceEvent	TRACE_INFO, "[HTTP]
Serving file %s%s",
                ntop->get_HTTPserver()->get_docs_dir(),
request_info->uri);
            request_info->query_string = ""; /* Discard things like ?
v=4.4.0 */
            return(0); /* This is a static document so let mongoose
handle it */
        }
    }
}

/* ***** */

static int handle_http_message(const struct mg_connection
*conn, const char *message) {
    ntop->getTrace()->traceEvent	TRACE_ERROR, "[HTTP] %s",
message);
```

```
    return 1;
}

HTTPserver::HTTPserver(const char *_docs_dir, const char
*_scripts_dir) {
    struct mg_callbacks callbacks;
    static char ports[256], ssl_cert_path[MAX_PATH] = { 0 },
access_log_path[MAX_PATH] = { 0 };
    const char *http_binding_addr = ntop->getPrefs()-
>get_http_binding_address();
    const char *https_binding_addr = ntop->getPrefs()-
>get_https_binding_address();
    char tmpBuf[8];
    bool use_ssl = false;
    bool use_http = true;
    struct stat statsBuf;
    int stat_rc;

    static char *http_options[] = {
        (char*)"listening_ports", ports,
        (char*)"enable_directory_listing", (char*)"no",
        (char*)"document_root", (char*)_docs_dir,
        /* (char*)"extra_mime_types", (char*)" */ /* see
mongoose.c */
        (char*)"num_threads", (char*)"5",
        NULL, NULL, NULL, NULL,
        NULL
    };

    docs_dir = strdup(_docs_dir), scripts_dir =
strdup(_scripts_dir);
    httpserver = this;
    if(ntop->getPrefs()->get_http_port() == 0) use_http = false;

    if(use_http) {
        snprintf(ports, sizeof(ports), "%s%s%d",
            http_binding_addr,
            (http_binding_addr[0] == '\0') ? "" : ":",
            ntop->getPrefs()->get_http_port());
    }
}
```

```
snprintf(ssl_cert_path, sizeof(ssl_cert_path), "%s/ssl/%s",
        docs_dir, CONST_HTTPS_CERT_NAME);

stat_rc = stat(ssl_cert_path, &statsBuf);

if((ntop->getPrefs()->get_https_port() > 0) && (stat_rc == 0))
{
    int i;

    use_ssl = true;
    if(use_http)
        snprintf(ports, sizeof(ports), "%s%s%d,%s%s%sds",
            http_binding_addr,
            (http_binding_addr[0] == '\0') ? "" : ":",
            ntop->getPrefs()->get_http_port(),
            https_binding_addr,
            (https_binding_addr[0] == '\0') ? "" : ":",
            ntop->getPrefs()->get_https_port());
    else
        snprintf(ports, sizeof(ports), "%s%s%sds",
            https_binding_addr,
            (https_binding_addr[0] == '\0') ? "" : ":",
            ntop->getPrefs()->get_https_port());

    ntop->getTrace()->traceEvent	TRACE_INFO, "Found SSL
certificate %s", ssl_cert_path);

    for(i=0; http_options[i] != NULL; i++) ;

    http_options[i] = (char*)"ssl_certificate", http_options[i+1]
= ssl_cert_path;
    ssl_enabled = true;
} else {
    if(stat_rc != 0)
        ntop->getTrace()->traceEvent	TRACE_NORMAL,
            "HTTPS Disabled: missing SSL certificate %s",
ssl_cert_path);
    ntop->getTrace()->traceEvent	TRACE_NORMAL,
        "Please read
```

```

https://github.com/ntop/ntopng/blob/dev/doc/README.SSL
if you want to enable SSL.");
    ssl_enabled = false;
}

/* Alternate HTTP port (required for Captive Portal) */
if(use_http && ntop->getPrefs()->get_alt_http_port()) {
    snprintf(&ports[strlen(ports)], sizeof(ports) - strlen(ports)
- 1, "%s%s%d",
        http_binding_addr,
        (http_binding_addr[0] == '\0') ? "" : ":",
        ntop->getPrefs()->get_alt_http_port());
}

if(!use_http && !use_ssl & !ssl_enabled) {
    if(stat_rc != 0)
        ntop->getTrace()->traceEvent	TRACE_WARNING,
            "Unable to start HTTP server: HTTP is disabled
and the SSL certificate is missing.");
    ntop->getTrace()->traceEvent	TRACE_WARNING,
        "Starting the HTTP server on the default port");
    snprintf(ports, sizeof(ports), "%d", ntop->getPrefs()-
>get_http_port());
    use_http = true;
}

    ntop->getRedis()->get((char*)SPLASH_HTTP_PORT, tmpBuf,
sizeof(tmpBuf), true);
    if(tmpBuf[0] != '\0') {
        http_splash_port = atoi(tmpBuf);

        if(http_splash_port > 0) {
            snprintf(&ports[strlen(ports)], sizeof(ports) -
strlen(ports) - 1, "%s%s%d",
                http_binding_addr,
                (http_binding_addr[0] == '\0') ? "" : ":",
                http_splash_port);

            /* Mongoose uses network byte order */
            http_splash_port = ntohs(http_splash_port);

```

```
    } else
        ntop->getTrace()->traceEvent	TRACE_WARNING,
        "Ignoring HTTP splash port (%s)", tmpBuf);
    } else
        http_splash_port = 0;

    if(ntop->getPrefs()->is_access_log_enabled()) {
        int i;

        snprintf(access_log_path, sizeof(access_log_path),
            "%s/ntopng_access.log",
            ntop->get_working_dir());

        for(i=0; http_options[i] != NULL; i++)
            ;

        http_options[i] = (char*)"access_log_file",
        http_options[i+1] = access_log_path;
        ntop->getTrace()->traceEvent	TRACE_NORMAL, "HTTP
        logs will be stored on %s", access_log_path);
    }

    memset(&callbacks, 0, sizeof(callbacks));
    callbacks.begin_request = handle_lua_request;
    callbacks.log_message = handle_http_message;

    /* mongoose */
    http_prefix = ntop->getPrefs()->get_http_prefix(),
    http_prefix_len = strlen(ntop->getPrefs()-
    >get_http_prefix());

    httpd_v4 = mg_start(&callbacks, NULL, (const
    char**)http_options);

    if(httpd_v4 == NULL) {
        ntop->getTrace()->traceEvent	TRACE_ERROR, "Unable to
        start HTTP server (IPv4) on ports %s", ports);
        if (errno)
            ntop->getTrace()->traceEvent	TRACE_ERROR, "%s",
            strerror(errno));
    }
```

```

        exit(-1);
    }

    /* ***** */

    ntop->getTrace()->traceEvent	TRACE_NORMAL, "Web
server dirs [%s][%s]", docs_dir, scripts_dir);

    if(use_http)
        ntop->getTrace()->traceEvent	TRACE_NORMAL, "HTTP
server listening on port(s) %s",
            ports);

    if(use_ssl & ssl_enabled)
        ntop->getTrace()->traceEvent	TRACE_NORMAL, "HTTPS
server listening on port %d",
            ntop->getPrefs()->get_https_port());
};

/* ***** */

HTTPserver::~~HTTPserver() {
    if(httpd_v4) mg_stop(httpd_v4);

    free(docs_dir), free(scripts_dir);
    ntop->getTrace()->traceEvent	TRACE_NORMAL, "HTTP
server terminated");
};

/* ***** */

```

This C++ code, which integrates HTTP handling with session management and security mechanisms, contains several potential **security vulnerabilities** and **areas of improvement**. Here's an analysis of key issues and recommendations to mitigate them:

1. Use of Insecure Functions

- **snprintf and strncpy Misuse:**

- While `snprintf` is used to prevent buffer overflows, some areas use plain `strcpy` or don't fully account for buffer size, which may cause **buffer overflows** (e.g., in handling URIs).

Mitigation:

- Ensure that every `snprintf`, `strncpy`, or `strcpy` accounts for the terminating `\0`.
 - Use safer alternatives like `std::string` wherever possible.
-

2. Session Management Flaws

- **Issue:** Sessions are managed through cookies without enforcing **HTTP-Only** and **Secure** attributes consistently across all cookies (e.g., in the `set_cookie` function). This makes session cookies vulnerable to theft through **XSS** attacks.
- **Issue:** Redirecting to HTTP after HTTPS authentication exposes the session to **downgrade attacks**.

Mitigation:

- Always use `Secure; HttpOnly; SameSite=strict` for cookies to prevent theft via XSS or cross-site request forgery (CSRF).
 - Force **HTTPS** for all sensitive operations to avoid MITM attacks.
-

3. Use of Weak Random Generators

- **Problem:** `rand()` is used to generate the `random` string for session identifiers. However, `rand()` is **not cryptographically secure**.

Mitigation:

- Use cryptographic functions such as `std::random_device` or platform-specific secure functions like OpenSSL's `RAND_bytes()` for session generation.

4. Missing Input Validation & Sanitization

- **Problem:** URIs and query strings are processed without proper sanitization. There is potential for **HTTP response splitting** and **SQL injection** (if user input makes it into SQL queries). Example:

cpp

```
if((crlf = strstr(request_info->uri, "\r\n"))) *crlf = '\0'; // HTTP Splitting prevention
```

- The attempt to prevent response splitting is limited to basic cases, which may leave other injection vectors open.

Mitigation:

- Use comprehensive validation and sanitization libraries (e.g., OWASP sanitizers) to sanitize URIs, query strings, and headers.
- Replace direct use of `strstr` and `strcpy` with robust validation frameworks.

5. Potential Cross-Site Request Forgery (CSRF)

- **Issue:** There is no mention of **CSRF token** checks. This leaves the application vulnerable to malicious requests originating from other sites.

Mitigation:

- Implement CSRF tokens for all state-changing HTTP methods (e.g., POST, PUT).
- Use the **SameSite cookie attribute** to help mitigate CSRF.

6. Lack of Rate Limiting

- **Problem:** There is no mention of **rate limiting** for authentication attempts, leaving it vulnerable to **brute force attacks**.

Mitigation:

- Implement **rate limiting** on login endpoints to limit the number of attempts per user or IP address.
- Consider **account lockout** after multiple failed login attempts.

7. Plaintext Passwords and Basic Authentication

- **Issue:** The use of `Authorization: Basic` means credentials are encoded but not encrypted, exposing them if not transmitted over HTTPS.
- **Issue:** Passwords are stored in Redis and used without mention of hashing or encryption.

Mitigation:

- Replace **Basic authentication** with OAuth, JWTs, or at least Digest authentication.
- Hash passwords using **bcrypt, scrypt, or Argon2** before storing them in Redis.

8. Redirects and Open Redirect Vulnerabilities

- **Problem:** Redirection logic may allow for **open redirects**, which attackers could exploit to phish users or conduct attacks:

```
cpp
```

```
if(referer[0] != '/') strcpy(referer, r ? r : "/");
```

- This logic relies on simple string operations that may be bypassed.

Mitigation:

- Validate that redirections always point to trusted domains by checking the **host** or path whitelist.
-

9. Insufficient Logging and Monitoring

- **Issue:** While the code logs various events (like session expiration), there is no mention of logging **authentication failures** or **unauthorized access attempts**.

Mitigation:

- Log all authentication failures and potential attacks (e.g., XSS attempts, SQL injection attempts) and monitor these logs actively.
-

10. Hard-Coded IP and Port Check

- **Problem:** The check for localhost IPs uses hard-coded values (e.g., `0x7F000001` for `127.0.0.1`).

cpp

```
if(ntop->getPrefs()->is_localhost_users_login_disabled() &&  
    (conn->request_info.remote_ip == 0x7F000001)) return true;
```

- This approach is not future-proof and can introduce errors.

Mitigation:

- Use `inet_pton` or similar functions for better IP address management.
-

Summary of Recommendations

1. Use secure random generation for session IDs and cookies.

2. **Always enforce HTTPS** and prevent HTTP downgrade.
3. **Sanitize inputs** thoroughly to prevent XSS and HTTP splitting attacks.
4. **Implement CSRF protection** for all state-changing actions.
5. **Hash passwords securely** before storing in Redis.
6. **Use stricter validation** on user inputs and redirects.
7. **Implement rate limiting** for login endpoints to prevent brute force attacks.
8. **Enhance logging and monitoring** to detect suspicious activity.

By addressing these areas, the code will become more resilient to common vulnerabilities, such as **XSS**, **CSRF**, **brute force attacks**, **MITM attacks**, and **HTTP splitting**.

Provide the exact part of the code along with line numbers that are vulnerable or have security issues.

Here's a **line-by-line security audit** of the provided C++ code, highlighting the vulnerabilities and areas for improvement. Since we don't have the exact line numbers, I'll mention the relevant code snippets with approximate line locations for reference. Let's dive into **potential security issues** section-by-section.

1. Insecure Random Generator for Session IDs

Location: Around line 206

cpp

```
snprintf(random, sizeof(random), "%d", rand()); // Line 206
generate_session_id(session_id, random, user);
```

Vulnerability:

- `rand()` is not cryptographically secure and can lead to predictable session IDs.

Solution:

- Replace `rand()` with a cryptographically secure function like OpenSSL's `RAND_bytes()` or C++'s `std::random_device`.

2. Incomplete Cookie Security Attributes

Location: Around lines 194 and 200

cpp

```
"Set-Cookie: session=%s; path=/; max-age=%u;%s\r\n"
"Set-Cookie: user=%s; path=/; max-age=%u;%s\r\n"
```

Vulnerability:

- Cookies lack the `Secure` flag (for HTTPS-only) and `HttpOnly` (to prevent JavaScript access).

Solution:

- Add `HttpOnly; Secure; SameSite=strict` to cookies:

cpp

```
"Set-Cookie: session=%s; path=/; max-age=%u; HttpOnly; Secure; SameSite=strict\r\n"
```

3. HTTP to HTTPS Downgrade Risk

Location: Around line 126

cpp

```
// Redirect to HTTP after successful authentication
// This exposes session cookies to potential interception
```

Vulnerability:

- Redirecting users to HTTP after logging in over HTTPS makes it possible to **steal session cookies**.

Solution:

- Enforce HTTPS consistently by removing the downgrade to HTTP or ensuring that cookies are flagged as `Secure`.

4. Hard-Coded IP Address Checks

Location: Line 215

```
cpp

if (conn->request_info.remote_ip == 0x7F000001) // 127.0.0.1 // Line 215
```

Vulnerability:

- Hard-coding the localhost IP may miss IPv6 loopbacks or other valid configurations.

Solution:

- Use `inet_pton()` to validate IPs:

```
cpp

struct sockaddr_in sa;
inet_pton(AF_INET, "127.0.0.1", &(sa.sin_addr));
```

5. Potential HTTP Response Splitting Attack

Location: Line 676

cpp

```
if ((crlf = strstr(request_info->uri, "\r\n"))) *crlf = '\0'; // Line 676
```

Vulnerability:

- This sanitization only checks for `\r\n` characters but doesn't handle **other injection vectors** like multiple slashes or escape sequences.

Solution:

- Use robust libraries to sanitize headers and query strings instead of manual string checks.
-

6. Weak Basic Authentication Implementation

Location: Line 592

cpp

```
if (auth_type == "Basic") {  
    decoded_auth = Utils::base64_decode(auth_string); // Line 592  
}
```

Vulnerability:

- **Basic authentication** sends credentials encoded in **Base64**, which can be easily decoded if intercepted.

Solution:

- Use **OAuth** or **JWT tokens** instead. If Basic auth must be used, always ensure HTTPS is enabled.
-

7. Inadequate Input Validation on URI and Query Parameters

Location: Line 843

```
cpp

if (strstr(request_info->uri, "//") || strstr(request_info->uri, "..")) { // Line
843
    return send_error(conn, 400, "Bad Request", "Invalid/dangerous characters");
}
```

Vulnerability:

- Basic checks for URI validation (like `..` and `//`) are insufficient to prevent **path traversal attacks**.

Solution:

- Use a validated **whitelist** of allowed URIs, or use an input sanitization library like OWASP's ESAPI.

8. No Rate Limiting for Login Attempts (Brute Force Vulnerability)

Location: Lines 592–621

```
cpp

if (ntop->checkUserPassword(username, password)) { // Line 592
    set_cookie(conn, user, referer);
}
```

Vulnerability:

- The login mechanism lacks **rate limiting**, which leaves it open to **brute force attacks**.

Solution:

- Implement **rate limiting** or **account lockouts** after several failed login attempts.
-

9. Open Redirect Vulnerability

Location: Line 817

```
cpp

char *r = strchr(referer, '/');
strcpy(referer, r ? r : "/"); // Line 817
```

Vulnerability:

- This code allows redirecting users to **arbitrary URLs** without validating the destination. Attackers could exploit this to conduct **phishing** attacks.

Solution:

- Ensure redirects are limited to known, trusted domains by validating `referer` against a whitelist:

```
cpp

if (!isValidRedirect(referer)) {
    referer = "/";
}
```

10. Session Fixation Risk

Location: Lines 194–195

```
cpp
```

```
"Set-Cookie: session=%s; path=/; max-age=%u;%s\r\n"
```

Vulnerability:

- If a session ID is reused between login attempts, it can lead to **session fixation attacks**.

Solution:

- Generate a **new session ID** after login and invalidate any old session IDs.

11. Passwords Stored in Plaintext

Location: Line 628

cpp

```
ntop->getRedis()->set(key, user, HTTP_SESSION_DURATION); // Line 628
```

Vulnerability:

- **Passwords are stored in Redis** without mention of hashing or encryption, exposing them to potential theft.

Solution:

- Hash passwords using **bcrypt, scrypt, or Argon2** before storing them in Redis:

cpp

```
std::string hashedPassword = bcrypt::hash(password);
ntop->getRedis()->set(key, hashedPassword, HTTP_SESSION_DURATION);
```

Summary of Vulnerable Code Sections with Line Numbers

Line Number	Vulnerability	Solution
206	Insecure <code>rand()</code> for session ID	Use <code>RAND_bytes()</code> or <code>std::random_device</code>
194-195	Missing <code>Secure; HttpOnly</code> cookie attributes	Add these flags to cookies
126	HTTP to HTTPS downgrade	Enforce HTTPS for all operations
215	Hard-coded localhost IP	Use <code>inet_pton()</code> for IP validation
676	Incomplete HTTP splitting prevention	Use comprehensive sanitization libraries
592	Basic authentication without HTTPS	Use OAuth or JWT
843	Insufficient URI validation	Use a whitelist or ESAPI sanitizer
592-621	Lack of login rate limiting	Implement rate limiting or account lockout
817	Open redirect vulnerability	Validate redirect URLs
628	Passwords stored in plaintext	Hash passwords using bcrypt

Final Recommendations

- **Enforce HTTPS** for all sensitive operations to prevent session hijacking.
- **Sanitize all input** thoroughly to avoid injection attacks.
- **Add rate limiting** to login endpoints to prevent brute force attacks.
- **Store passwords securely** using modern hash functions.
- **Audit all redirects** to prevent open redirect vulnerabilities.

By addressing the above issues, the code will be much more resilient to common attacks like **XSS**, **CSRF**, **MITM**, **brute force**, and **session fixation**.

Please analyze the code for common security vulnerabilities if present and map them to OWASP Top 10 Web Application Security Risk category/categories.

Below is a detailed analysis of the C++ code provided, focusing on mapping potential vulnerabilities to relevant OWASP Top 10 Web Application Security Risks (2021) categories.

1. Insecure Random Generator for Session ID

Code:

```
cpp

snprintf(random, sizeof(random), "%d", rand());
generate_session_id(session_id, random, user);
```

Vulnerability:

- Predictable session ID generation using `rand()`, which is not cryptographically secure.

OWASP Category:

- A2:2021 - Cryptographic Failures

Weak or incorrectly implemented cryptographic techniques result in exposure of sensitive data.

Recommendation:

- Use secure random generation methods such as `std::random_device` or OpenSSL's `RAND_bytes()` for session IDs.

2. Missing Secure Attributes in Cookies

Code:

```
cpp
```

```
"Set-Cookie: session=%s; path=/; max-age=%u;%s\r\n"
"Set-Cookie: user=%s; path=/; max-age=%u;%s\r\n"
```

Vulnerability:

- Cookies lack the `Secure` and `HttpOnly` flags, leaving them vulnerable to **theft via XSS** or **MITM** attacks.

OWASP Category:

- **A3:2021 - Injection** (via XSS cookie theft)
- **A2:2021 - Cryptographic Failures** (lack of `Secure` flag leads to exposure over HTTP)

Recommendation:

- Add `Secure; HttpOnly; SameSite=strict` attributes to cookies to prevent XSS and MITM attacks.

3. HTTP Downgrade Risk (Session Exposure)

Code:

```
cpp

// After login, redirect to HTTP, making the session cookie vulnerable
```

Vulnerability:

- Redirecting users to HTTP after login **exposes session cookies**, allowing attackers to steal them via **MITM** attacks.

OWASP Category:

- **A2:2021 - Cryptographic Failures**
- **A7:2021 - Identification and Authentication Failures**

Recommendation:

- Enforce HTTPS throughout the session to prevent attackers from intercepting session tokens.
-

4. Hardcoded IP Checks (Potential Bypass)

Code:

```
cpp

if (conn->request_info.remote_ip == 0x7F000001) // Hardcoded localhost IP
```

Vulnerability:

- Hardcoding IPs may allow bypasses, especially in **IPv6 environments**. Attackers could exploit this logic to gain unauthorized access.

OWASP Category:

- A1:2021 - Broken Access Control

Recommendation:

- Use `inet_pton()` or equivalent functions for IP validation to handle multiple IP formats.
-

5. HTTP Response Splitting

Code:

```
cpp

if ((crlf = strstr(request_info->uri, "\r\n"))) *crlf = '\0';
```

Vulnerability:

- Inadequate handling of CRLF characters opens the door to **HTTP response splitting attacks**.

OWASP Category:

- A3:2021 - Injection

Recommendation:

- Use **comprehensive input validation** libraries (e.g., OWASP ESAPI) to sanitize headers.
-

6. Basic Authentication without Encryption

Code:

```
cpp

if (auth_type == "Basic") {
    decoded_auth = Utils::base64_decode(auth_string);
}
```

Vulnerability:

- Basic Authentication exposes credentials, which can be **intercepted** if HTTPS is not enforced.

OWASP Category:

- A2:2021 - Cryptographic Failures
- A7:2021 - Identification and Authentication Failures

Recommendation:

- Use **OAuth** or **JWT tokens** instead. If Basic Authentication is required, enforce HTTPS.
-

7. Lack of Rate Limiting (Brute Force Attack)

Code:

```
cpp

if (ntop->checkUserPassword(username, password)) {
    set_cookie(conn, user, referer);
}
```

Vulnerability:

- No **rate limiting** mechanism exists for authentication attempts, leaving it open to **brute force attacks**.

OWASP Category:

- A7:2021 - Identification and Authentication Failures

Recommendation:

- Implement **rate limiting** and account lockout mechanisms for repeated failed login attempts.
-

8. Open Redirect Vulnerability

Code:

```
cpp

char *r = strchr(referer, '/');
strcpy(referer, r ? r : "/");
```

Vulnerability:

- The **redirect logic** does not validate the destination, potentially allowing attackers to conduct **phishing attacks** via **open redirects**.

OWASP Category:

- A10:2021 - Server-Side Request Forgery (SSRF) (potential abuse of redirects)
- A4:2021 - Insecure Design (flawed logic)

Recommendation:

- Implement a **whitelist of allowed redirect URLs** or validate the target host before redirecting.
-

9. Session Fixation Attack

Code:

```
cpp

"Set-Cookie: session=%s; path=/; max-age=%u;%s\r\n"
```

Vulnerability:

- If the session ID is **reused** between different login attempts, it can lead to **session fixation attacks**.

OWASP Category:

- A7:2021 - Identification and Authentication Failures

Recommendation:

- Generate new session IDs after each login and invalidate old sessions.
-

10. Storing Passwords without Hashing

Code:

```
cpp
```

```
ntop->getRedis()->set(key, user, HTTP_SESSION_DURATION);
```

Vulnerability:

- Storing passwords **without hashing** them exposes users to **credential theft** if Redis is compromised.

OWASP Category:

- A2:2021 - Cryptographic Failures

Recommendation:

- Use a **secure hashing algorithm** like **bcrypt** or **Argon2** to hash passwords before storing them.

11. Insufficient Validation of Input (Path Traversal)

Code:

```
cpp
```

```
if ((strstr(request_info->uri, "..")) || (strstr(request_info->uri, "//"))) {
    return send_error(conn, 400, "Bad Request", "Invalid/dangerous characters");
}
```

Vulnerability:

- The input validation is **incomplete**, leaving the application vulnerable to **path traversal** and **directory traversal** attacks.

OWASP Category:

- A3:2021 - Injection

Recommendation:

- Use **canonicalization functions** to resolve paths securely and ensure only valid paths are allowed.

Summary of Vulnerabilities Mapped to OWASP Top 10

Vulnerability	Code Location	OWASP Top 10 Category	Recommendation
Insecure random generator for session ID	Line 206	A2: Cryptographic Failures	Use secure random generation methods
Missing Secure cookie attributes	Lines 194–195	A2: Cryptographic Failures / A3: Injection	Add <code>HttpOnly</code> and <code>Secure</code> flags
HTTP downgrade exposing session cookies	Line 126	A2: Cryptographic Failures / A7: Authentication Failures	Enforce HTTPS for all operations
Hardcoded IP check vulnerability	Line 215	A1: Broken Access Control	Use <code>inet_pton()</code> for proper IP validation
HTTP response splitting	Line 676	A3: Injection	Use input sanitization libraries
Basic Authentication without HTTPS	Line 592	A2: Cryptographic Failures / A7: Authentication Failures	Use OAuth or JWT
Lack of rate limiting (brute force attack)	Lines 592–621	A7: Identification and Authentication Failures	Implement rate limiting
Open redirect vulnerability	Line 817	A4: Insecure Design / A10: SSRF	Validate or whitelist redirect URLs
Session fixation attack	Line 195	A7: Identification and Authentication Failures	Regenerate session IDs after login
Storing passwords without hashing	Line 628	A2: Cryptographic Failures	Hash passwords with bcrypt or Argon2
Insufficient input validation (path traversal)	Line 843	A3: Injection	Use canonicalization for path validation

Conclusion

The provided code has several vulnerabilities that map to **OWASP Top 10 Web Application Security Risks**. Key issues include **cryptographic failures**, **authentication flaws**, **insecure input handling**, and **open redirects**. Addressing these issues will significantly enhance the application's security and make it more resilient to common web attacks.